Programación Multimedia y Dispositivos Móviles

Guía de estudio: proyectos de la carpeta ProgMov del repositorio DAM_2 y resolución de los exámenes *Six Nations* y *Planetas del Sistema Solar*.

Contenido

- **Parte 1.** Los proyectos del repositorio: qué hace cada uno, qué conceptos enseña y cuándo usarlos.
- **Parte 2.** Examen Six Nations (sixnations4): ListView personalizado, AlertDialog, menús y SharedPreferences.
- **Parte 3.** Examen Planetas del Sistema Solar: Splash con Thread, TableLayout, ordenación y SharedPreferences.
- **Parte 4.** Chuleta rápida: qué usar para cada cosa.

Estructura común de todas las Activity

Todas las actividades de los proyectos empiezan igual. Este bloque lo genera Android Studio y conviene dejarlo tal cual:

```
public class MainActivity extends AppCompatActivity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        EdgeToEdge.enable(this);                    // la app dibuja detrás de las barras del sistema
        setContentView(R.layout.activity_main);     // qué XML se muestra
        ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), (v, insets) -> {
            Insets systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars());
            v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom);
            return insets;                          // relleno para no quedar debajo de las barras
        });
        // ...aquí van los findViewById y los listeners
    }
}
```

- El layout raíz necesita **android:id="@+id/main"**, porque el bloque de arriba lo busca.
- **findViewById(R.id.x)** conecta una variable Java con un elemento del XML. Se hace después de setContentView.
- **Toast.makeText(getApplicationContext(), "texto", Toast.LENGTH_SHORT).show()** muestra un mensaje breve.

Parte 1. Proyectos del repositorio (ProgMov)

Están agrupados por tema. Dentro de cada grupo, de más simple a más completo. La carpeta **RevisarExamen** contiene copias de 07SpinnerPaises, 9TimePickerDialog, 9TimePickerDialog2, 11Internacionalizacion y 12IntentExplicita para repasar; funcionan igual que los originales.

1.1 Botones y eventos: las calculadoras

Hay tres formas de responder a un click. Las tres funcionan; cambia dónde se escribe la conexión.

Forma	Dónde se conecta	Proyecto	Cuándo
android:onClick en XML	En el XML: android:onClick="sumar". En Java: public void sumar(View view)	Calculadora2, CALCULADORAV10	Pantallas sencillas, rápido de escribir
setOnClickListener con lambda	En Java: btn.setOnClickListener(v -> sumar());	CALCULADORAV10_local, PanelInicioSesion	Forma recomendada: el XML no depende de nombres de métodos
implements View.OnClickListener	La Activity implementa onClick(View v) y distingue con v.getId()	CalculadoraProgramatica3, OrientacionLayoutBloqueo, 9TimePickerDialog2	Varios botones que comparten lógica

Calculadora2 / CALCULADORAV10

Qué hace: Calculadora de dos operandos con botones sumar, restar, multiplicar y dividir.

- El evento se declara en el XML con **android:onClick**; el método Java debe ser **public void** y recibir **View**.
- **Double.parseDouble(et.getText().toString())** convierte el texto del EditText en número.
- Dividir entre 0 se controla lanzando y capturando **ArithmeticException**.

Cuándo usarlo: cuando quieres el mínimo código y la pantalla es simple.

```
<Button android:text="+" android:onClick="sumar" ... />

public void sumar(View view) {
    operando1 = Double.parseDouble(etOperando1.getText().toString());
    operando2 = Double.parseDouble(etOperando2.getText().toString());
    resultado = operando1 + operando2;
    tvResultado.setText(resultado.toString());
}
```

CALCULADORAV10_local

Qué hace: La misma calculadora, pero sin android:onClick: los botones se conectan desde Java.

- Los botones se declaran como variables locales dentro de onCreate.
- **btnSumar.setOnClickListener(v -> sumar());** usa una lambda (forma corta de clase anónima).

Cuándo usarlo: es la forma más habitual hoy y la que conviene usar por defecto.

CalculadoraProgramatica3

Qué hace: Calculadora con RadioButton en lugar de botones: al marcar una operación se calcula.

- La Activity hace **implements View.OnClickListener** y registra **this** en los 4 RadioButton.
- Un único **onClick(View v)** pregunta **v.getId() == R.id.rbSumar**, etc.
- **try/catch NumberFormatException** avisa con Toast si falta un operando (texto vacío).

Cuándo usarlo: cuando muchos controles hacen cosas parecidas y quieres un solo punto de entrada.

```
@Override
public void onClick(View v) {
    try {
        operando1 = Double.parseDouble(etOperando1.getText().toString());
        operando2 = Double.parseDouble(etOperando2.getText().toString());
        if (v.getId() == R.id.rbSumar) sumar(operando1, operando2);
        if (v.getId() == R.id.rbRestar) restar(operando1, operando2);
    } catch (NumberFormatException e) {
        Toast.makeText(getApplicationContext(), "Falta operando", Toast.LENGTH_SHORT).show();
    }
}
```

PanelInicioSesion

Qué hace: Pantalla con botón Iniciar sesión (muestra "hola!") y botón Salir.

- Listeners con lambda que llaman a métodos propios (**login()**, **salir()**).
- **finishAffinity()** cierra esta actividad y todas las de la app que estén debajo.

Cuándo usarlo: botón "Salir" de la aplicación completa.

1.2 Controles de selección

HotelComidas

Qué hace: Tres CheckBox (desayuno 10 €, comida 25 €, cena 30 €) que suman o restan al total a pagar.

- La Activity hace **implements CompoundButton.OnCheckedChangeListener**.
- **onCheckedChanged(CompoundButton buttonView, boolean isChecked)** salta al marcar y al desmarcar.
- **buttonView.getId()** dice qué CheckBox cambió; **isChecked()** si quedó marcado.

Cuándo usarlo: opciones que se pueden marcar varias a la vez (no excluyentes).

```
if (buttonView.getId() == R.id.cbDesayuno) {
    if (cbDesayuno.isChecked()) totalPrecio += 10;
    else totalPrecio -= 10;
    tvTotalPagar.setText("Total a pagar: " + totalPrecio + "€");
}
```

Hamburgueseria

Qué hace: Interfaz de pedido con RadioButton (ternera, pollo, pescado), CheckBox (pepinillos, queso, bacon), un Switch de descuento y un ImageView.

- Solo tiene los **findViewById**; la lógica no está hecha todavía.
- **RadioButton** dentro de un **RadioGroup** = solo uno marcado a la vez (excluyentes).

Cuándo usarlo: como ejemplo de combinar los distintos controles en un formulario.

Toggle_SwitchButton

Qué hace: Un ToggleButton y un Switch que cambian el texto y el color de un TextView (Activar / Desactivar).

- **setOnCheckedChangeListener** igual que en los CheckBox.
- Lee recursos desde Java: **getResources().getText(R.string.activar)** y **getResources().getColor(R.color.rojo)**.
- Los textos están en **res/values/strings.xml** y los colores en **res/values/colors.xml**.

Cuándo usarlo: interruptores de encendido/apagado (sonido, notificaciones, modo oscuro).

1.3 Layouts, formularios y configuración

TableLayoutFormulario

Qué hace: Formulario de producto (código, nombre, precio) colocado en un TableLayout, con botones Guardar y Limpiar.

- **TableLayout** + **TableRow**: organiza la pantalla en filas y columnas.
- Validación: **texto.trim().isEmpty()** y Toast "Por favor, completa todos los campos".
- **et.setText("")** limpia y **et.requestFocus()** devuelve el cursor al primer campo.

Cuándo usarlo: formularios con etiquetas y campos alineados; patrón de validación para cualquier formulario.

```
if (codigo.isEmpty() || nombre.isEmpty() || precio.isEmpty())
    Toast.makeText(this, "Por favor, completa todos los campos", Toast.LENGTH_SHORT).show();
else { /* guardar */ }
```

OrientacionLayoutBloqueo

Qué hace: Tres botones que fuerzan la orientación: horizontal, vertical o automática.

- **setRequestedOrientation(ActivityInfo.SCREEN_ORIENTATION_LANDSCAPE)** / **PORTRAIT** / **SENSOR**.
- También se puede fijar en el Manifest con **android:screenOrientation**.

Cuándo usarlo: apps o pantallas que solo tienen sentido en una orientación (vídeo, juegos, formularios).

11Internacionalizacion

Qué hace: Formulario cuyos textos cambian de idioma según el idioma del móvil.

- Textos en **res/values/strings.xml** (por defecto, español), **values-en** (inglés) y **values-fr-rFR** (francés de Francia).
- En el XML se usa **android:text="@string/nombre"** en vez del texto escrito a mano.
- Android elige solo la carpeta que coincide con el idioma del sistema.

Cuándo usarlo: siempre que la app deba estar en varios idiomas. Buena práctica incluso con uno solo.

1.4 Spinner (desplegable) y banderas

07SpinnerPaises

Qué hace: Spinner con países; al elegir uno se muestra su bandera y su población, y al tocar la bandera se abre Wikipedia.

- **Arrays paralelos**: nombres[], banderas[], poblaciones[], urls[]. La misma posición = el mismo país.
- La posición 0 es "Seleccione país" (sin bandera).
- **ArrayAdapter** conecta el array con el Spinner.
- **onItemSelected(..., int position, ...)** se ejecuta al elegir; con **position** se accede a los arrays.
- **Intent implícito**: new Intent("android.intent.action.VIEW", Uri.parse(url)) abre el navegador.

Cuándo usarlo: elegir una opción de una lista fija y mostrar datos asociados. Es la base del examen Six Nations.

```
String []nombres  = {"Seleccione país", "Alemania", "China", ...};
int    []banderas = {0, R.drawable.alemania, R.drawable.china, ...};

ArrayAdapter<String> arrayAdapter = new ArrayAdapter<>(getApplicationContext(),
        androidx.appcompat.R.layout.support_simple_spinner_dropdown_item, nombres);
spPaises.setAdapter(arrayAdapter);
spPaises.setOnItemSelectedListener(new AdapterView.OnItemSelectedListener() {
    public void onItemSelected(AdapterView<?> parent, View view, int position, long id) {
        imgvBanderas.setImageResource(banderas[position]);
    }
    public void onNothingSelected(AdapterView<?> parent) { }
});
```

Banderas / SpinnerBanderas

Qué hace: Banderas es una versión anterior y más simple del mismo ejercicio (España y Francia). SpinnerBanderas es la plantilla vacía, sin código.

- Mismo patrón: arrays paralelos + ArrayAdapter + onItemSelected + setImageResource.

Cuándo usarlo: como versión mínima para entender el Spinner antes de 07SpinnerPaises.

1.5 Diálogos y navegación

9TimePickerDialog / 9TimePickerDialog2

Qué hace: Un botón abre un selector de hora y la hora elegida se escribe en un TextView.

- **Calendar.getInstance()** da la hora actual para que el diálogo empiece en ella.
- **new TimePickerDialog(contexto, listener, hora, minutos, true)**; true = formato 24 h. Se muestra con **.show()**.
- **onTimeSet** recibe la hora elegida; si minute < 10 se añade un 0 delante.
- Versión 1: listener como clase anónima. Versión 2: la Activity implementa View.OnClickListener.

Cuándo usarlo: pedir una hora (alarmas, citas). Existe también **DatePickerDialog** para fechas.

12IntentExplicita

Qué hace: La primera pantalla pide un nombre y abre una segunda que saluda "Hola nombre!!!". El botón Salir cierra la app.

- **Intent explícito**: new Intent(getApplicationContext(), SegundaActividad.class).
- **intent.putExtra("nombre", valor)** envía datos; **startActivity(intent)** abre la pantalla.
- En la segunda: **Bundle bundle = getIntent().getExtras()** y **bundle.getString("nombre")**.
- Cada Activity nueva debe estar declarada en el **AndroidManifest.xml**.

Cuándo usarlo: navegar entre pantallas pasando datos. Lo usa el examen de Planetas.

```
// Enviar
Intent intent = new Intent(getApplicationContext(), SegundaActividad.class);
intent.putExtra("nombre", cadenaNombre);
startActivity(intent);

// Recibir (en SegundaActividad)
Bundle bundle = getIntent().getExtras();
if (bundle != null) tvSaludo.setText("Hola " + bundle.getString("nombre") + "!!!");
```

1.6 Listas y menús

ListViewDeStrings

Qué hace: Agenda con un ListView de nombres. Click corto muestra el teléfono; pulsación larga muestra el sexo.

- **ArrayAdapter<String>** + **lv.setAdapter()**: cada String es una fila.
- **setOnItemClickListener**: click corto, recibe la **position** de la fila.
- **setOnItemLongClickListener**: devuelve boolean. **return true** consume el evento; con false también saltaría el click normal.

Cuándo usarlo: listas sencillas donde cada fila es un solo texto. Lo usa el examen de Planetas.

ListViewAdaptadorPersonalizado

Qué hace: Agenda de contactos con foto, nombre, descripción y código en cada fila; al pulsar se abre una pantalla de detalle.

- **Clase modelo** (ModeloPersona): agrupa los datos de cada elemento con getters y setters.
- **Adaptador propio** (AdaptadorPersona extends ArrayAdapter<ModeloPersona>): sobrescribe **getView**, que pinta cada fila.
- **item.xml**: diseño de UNA fila. Se infla con LayoutInflater y se reutiliza (convertView).
- Dentro de getView los elementos se buscan con **item.findViewById**, no con el de la Activity.
- Al pulsar, Intent explícito con putExtra a DetallePersona.

Cuándo usarlo: cuando cada fila tiene varios datos o imágenes. Es la base del examen Six Nations.

```
@Override
public View getView(int position, View convertView, ViewGroup parent) {
    View item = convertView;
    if (item == null) item = LayoutInflater.from(context).inflate(R.layout.item, parent, false);
    TextView tvNombre = item.findViewById(R.id.tvNombre);
    ModeloPersona persona = personas.get(position);
    tvNombre.setText(persona.getNombre());
    return item;
}
```

MenuAplicacion

Qué hace: Menú de la barra superior con dos opciones que muestran un Toast.

- El menú se define en **res/menu/menu_principal.xml** con un <item> por opción.
- **onCreateOptionsMenu**: lo infla con MenuInflater. **onOptionsItemSelected**: qué hacer según **item.getItemId()**.
- El tema debe **tener ActionBar** (Theme.Material3.DayNight, no NoActionBar); si no, el menú no aparece.

Cuándo usarlo: acciones generales de la pantalla (ordenar, ajustes, ayuda). Lo usan los dos exámenes.

```
@Override
public boolean onCreateOptionsMenu(Menu menu) {
    getMenuInflater().inflate(R.menu.menu_principal, menu);
    return true;
}
@Override
public boolean onOptionsItemSelected(@NonNull MenuItem item) {
    if (item.getItemId() == R.id.item1) { /* ... */ }
    return true;
}
```

Parte 2. Examen Six Nations (sixnations4)

Sistema de valoración de jugadores de rugby. Se entregan dos proyectos: **SixNations4_SharedPreferences** y **SixNations4_SinSharedPreferences**. Son idénticos salvo la persistencia.

2.1 Qué pide el enunciado

- ListView en la pantalla principal y un botón con la imagen del sistema **ic_input_add**.
- Al pulsarlo, AlertDialog "Agregar Jugador": nombre, desplegable con las 6 naciones y valoración de 0 a 10.
- Al elegir nación, la imagen genérica cambia a la bandera. Si falta algo al pulsar Guardar, Toast avisando.
- Menú de aplicación para ordenar por puntuación.
- Menú emergente por ítem: **Eliminar** (con confirmación) y **Actualizar** (diálogo relleno).
- Persistencia con SharedPreferences manteniendo el orden de inserción.

2.2 Archivos del proyecto

Archivo	Para qué sirve
MainActivity.java	Pantalla principal: lista, botón +, menús, diálogos, ordenación y SharedPreferences.
ModeloJugador.java	Un jugador: nombre, pais (posición en el array), puntos y orden de inserción.
AdaptadorJugador.java	Adaptador personalizado: pinta cada fila con item_jugador.xml.
activity_main.xml	FrameLayout con el ListView y el ImageButton flotando abajo a la derecha.
item_jugador.xml	Diseño de una fila: nombre | país | puntos | bandera.
dialogo_jugador.xml	Formulario del AlertDialog (sirve para Agregar y Actualizar).
menu_principal.xml	Menú de la barra: Ordenar por puntuación y Orden de inserción.
menu_contextual.xml	Menú emergente: Eliminar y Actualizar.
drawable/*.xml	Banderas provisionales: bandera_generica, inglaterra, escocia, gales, irlanda, francia, italia.

2.3 Cómo se resuelve cada requisito

Requisito	Solución	Viene de
Filas con varios datos	ModeloJugador + AdaptadorJugador + item_jugador.xml	ListViewAdaptadorPersonalizado
Botón ic_input_add	ImageButton con android:src="@android:drawable/ic_input_add". El prefijo @android: coge el recurso del sistema	—
Nación y bandera	Arrays paralelos PAISES[] y BANDERAS[]; posición 0 = "Elija nacionalidad" con imagen genérica	07SpinnerPaises
AlertDialog con formulario	Se infla dialogo_jugador.xml y se pasa con builder.setView(vista)	Concepto de diálogo de 9TimePickerDialog
Validación con Toast	isEmpty(), pais == 0, try/catch de parseInt y rango 0-10	TableLayoutFormulario, CalculadoraProgramatica3
Ordenar por puntuación	Collections.sort con un Comparator de mayor a menor	MenuAplicacion (menú)

Requisito	Solución	Viene de
Menú Eliminar/Actualizar	registerForContextMenu + onCreateContextMenu + onContextItemSelected	Nuevo
Confirmar eliminación	AlertDialog con setTitle, setMessage y botones Cancelar/Confirmar	Nuevo
Refrescar la lista	adaptador.notifyDataSetChanged() tras cada cambio	—

2.4 El AlertDialog de Agregar / Actualizar

Un único método sirve para las dos cosas: **mostrarDialogoJugador(posicion)**. Con **-1** es un jugador nuevo (formulario vacío, título "Agregar Jugador"); con una posición válida se rellenan los campos con ese jugador y el título es "Actualizar Jugador".

```
View vista = getLayoutInflater().inflate(R.layout.dialogo_jugador, null);
EditText etNombre = vista.findViewById(R.id.etNombre);    // se busca DENTRO de la vista
// ... resto de elementos, Spinner con PAISES y listener que cambia la bandera

AlertDialog.Builder builder = new AlertDialog.Builder(MainActivity.this);
builder.setView(vista);
AlertDialog dialogo = builder.create();

btnGuardar.setOnClickListener(new View.OnClickListener() {
    public void onClick(View v) {
        if (nombre.isEmpty() || pais == 0 || textoValoracion.isEmpty()) {
            Toast.makeText(getApplicationContext(), "Debe rellenar ...", Toast.LENGTH_LONG).show();
            return;                     // NO se cierra el diálogo
        }
        // ... crear o modificar el jugador
        adaptador.notifyDataSetChanged();
        dialogo.dismiss();
    }
});
dialogo.show();
```

Por qué los botones van dentro del layout: los botones del propio Builder (setPositiveButton) cierran el diálogo siempre al pulsarlos. Poniendo Cancelar y Guardar en el XML controlamos nosotros cuándo se cierra (con dismiss()), así el diálogo sigue abierto si la validación falla.

2.5 Menú contextual (Eliminar / Actualizar)

Es como el menú de opciones, pero asociado a una vista. Se abre **manteniendo pulsado** un ítem del ListView.

```
registerForContextMenu(lvJugadores);       // en onCreate

@Override
public void onCreateContextMenu(ContextMenu menu, View v, ContextMenu.ContextMenuInfo menuInfo) {
    super.onCreateContextMenu(menu, v, menuInfo);
    getMenuInflater().inflate(R.menu.menu_contextual, menu);
}

@Override
public boolean onContextItemSelected(@NonNull MenuItem item) {
    AdapterView.AdapterContextMenuInfo info = (AdapterView.AdapterContextMenuInfo) item.getMenuInfo();
    int posicion = info.position;             // fila sobre la que se abrió el menú
    if (item.getItemId() == R.id.itemEliminar)   { confirmarEliminar(posicion); return true; }
    if (item.getItemId() == R.id.itemActualizar) { mostrarDialogoJugador(posicion); return true; }
    return super.onContextItemSelected(item);
}
```

La confirmación usa los botones del Builder: **setNegativeButton("Cancelar", null)** solo cierra, y **setPositiveButton("Confirmar", listener)** borra con **jugadores.remove(posicion)**.

2.6 Ordenar por puntuación

```
Collections.sort(jugadores, new Comparator<ModeloJugador>() {
    public int compare(ModeloJugador j1, ModeloJugador j2) {
        return j2.getPuntos() - j1.getPuntos();     // j2 - j1 = de mayor a menor
    }
});
adaptador.notifyDataSetChanged();
```

- **compare** devuelve negativo si j1 va antes, positivo si va después y 0 si son iguales.
- Collections.sort es **estable**: con los mismos puntos se mantiene el orden que ya tenían.
- Cada jugador guarda un campo **orden** (nº de inserción). La opción extra "Orden de inserción" ordena por él para volver a la lista original.

2.7 SharedPreferences (solo en la versión con SP)

SharedPreferences es un fichero clave-valor privado de la app. Sirve para datos pequeños. Como no guarda listas directamente, se guarda el número de jugadores y cada dato con una clave numerada:

```
numJugadores = 3
nombre_0 = "John Collins"    pais_0 = 1    puntos_0 = 8
nombre_1 = "Richard Monteau" pais_1 = 5    puntos_1 = 8
...
```

```
preferencias = getSharedPreferences("datos_sixnations", Context.MODE_PRIVATE);   // en onCreate

private void guardarDatos() {                 // tras agregar, actualizar o eliminar
    ArrayList<ModeloJugador> copia = new ArrayList<>(jugadores);
    ordenarPorInsercion(copia);               // se guarda SIEMPRE en orden de inserción
    SharedPreferences.Editor editor = preferencias.edit();
    editor.clear();
    editor.putInt("numJugadores", copia.size());
    for (int i = 0; i < copia.size(); i++) {
        editor.putString("nombre_" + i, copia.get(i).getNombre());
        editor.putInt("pais_" + i, copia.get(i).getPais());
        editor.putInt("puntos_" + i, copia.get(i).getPuntos());
    }
    editor.apply();
}

private void cargarDatos() {                  // en onCreate, antes de crear el adaptador
    int num = preferencias.getInt("numJugadores", 0);   // 0 si es la primera vez
    for (int i = 0; i < num; i++) { /* getString / getInt y jugadores.add(...) */ }
}
```

Por qué el país se guarda como número: con la posición sacamos el nombre (PAISES[pais]) y la bandera (BANDERAS[pais]). No se guarda el R.drawable de la bandera porque esos IDs pueden cambiar al recompilar y aparecerían banderas equivocadas al cargar.

Versión sin SP: es el mismo código sin preferencias, guardarDatos() ni cargarDatos(). La lista solo vive en memoria y al cerrar la app se pierde.

2.8 Antes de entregar

- Las banderas son provisionales (dibujos vectoriales). Copia las del profesor en res/drawable con los mismos nombres y borra el .xml del mismo nombre: no pueden existir dos recursos con el mismo nombre.
- Si no quieres la opción extra "Orden de inserción", bórrala de menu_principal.xml y de onOptionsItemSelected.
- Build, Clean Project antes de entregar.

Parte 3. Examen Planetas del Sistema Solar

App que muestra los planetas ordenados por distancia al Sol, tamaño u orden alfabético, en ascendente o descendente. Se entregan **PlanetasSistemaSolar_SharedPreferences** y **PlanetasSistemaSolar_SinSharedPreferences**.

3.1 Qué pide el enunciado

- Splash Screen con **planetas_splashscreen.jpg**, hecho obligatoriamente con la clase **Thread**.
- Pantalla principal con fondo **planetas_fondo.jpg**, título y tres botones en un **TableLayout**.
- Los tres botones abren la **misma** actividad, que muestra un ListView ordenado según el botón.
- Menú principal con "Ordenación Ascendente" y "Ordenación Descendente".
- SharedPreferences para recordar el tipo de ordenación al cerrar y abrir la app.

3.2 Archivos del proyecto

Archivo	Para qué sirve
SplashActivity.java	Actividad de arranque (LAUNCHER). Muestra la imagen 3 s con un Thread y abre MainActivity.
MainActivity.java	Fondo, título, 3 botones, menú Ascendente/Descendente y SharedPreferences.
ListadoPlanetas.java	Segunda actividad: recibe criterio y sentido, ordena y rellena el ListView.
ModeloPlaneta.java	Un planeta: nombre, distancia (long) y diámetro (double).
activity_splash.xml	ImageView a pantalla completa.
activity_main.xml	LinearLayout con fondo, TextView título y TableLayout con los botones.
activity_listado_planetas.xml	TextView título + ListView.
menu_principal.xml	Grupo de dos opciones excluyentes (checkableBehavior="single").
AndroidManifest.xml	Declara las 3 actividades; el Splash es LAUNCHER y lleva tema sin barra.
drawable/*.jpg	planetas_fondo.jpg y planetas_splashscreen.jpg provisionales.

3.3 Cómo se resuelve cada requisito

Requisito	Solución	Viene de
Splash con Thread	Thread con sleep(3000); en finally, Intent a MainActivity y finish()	12IntentExplicita
Botones en TableLayout	stretchColumns="*" y layout_column para colocar cada botón	TableLayoutFormulario
Un onClick para 3 botones	implements View.OnClickListener y v.getId()	CalculadoraProgramatica3
Misma actividad para todo	putExtra("criterio") y putExtra("ascendente")	12IntentExplicita
Menú de ordenación	res/menu con group checkableBehavior="single"	MenuAplicacion
Ordenar	ArrayList + Collections.sort con un Comparator por criterio	ListViewAdaptadorPersonalizado (modelo)
Orden contrario	Collections.reverse tras ordenar	—

Requisito	Solución	Viene de
ListView	ArrayAdapter con simple_list_item_1	ListViewDeStrings
Persistencia	getBoolean / putBoolean de "ascendente"	—

3.4 Splash Screen con Thread

```
Thread hilo = new Thread() {
    @Override
    public void run() {
        try {
            sleep(3000);                        // espera 3 segundos SIN bloquear la pantalla
        } catch (InterruptedException e) {
            e.printStackTrace();
        } finally {
            startActivity(new Intent(getApplicationContext(), MainActivity.class));
            finish();                           // si no, "atrás" volvería al splash
        }
    }
};
hilo.start();
```

En el Manifest el Splash tiene el intent-filter **MAIN / LAUNCHER** y el tema **Theme.PlanetasSistemaSolar.Splash** (NoActionBar). MainActivity y ListadoPlanetas usan el tema con barra, necesario para ver el menú.

Por qué un Thread: si hiciéramos sleep en el hilo principal (el de la interfaz), la pantalla se congelaría y la imagen ni se llegaría a pintar. El hilo aparte espera y la interfaz sigue funcionando.

3.5 TableLayout con los botones

```
<TableLayout android:stretchColumns="*" ...>      <!-- 3 columnas del mismo ancho -->
    <TableRow>
        <Button android:id="@+id/btnProximidad" android:layout_column="1" .../>  <!-- centro -->
    </TableRow>
    <TableRow android:layout_marginTop="40dp">
        <Button android:id="@+id/btnTamano"     android:layout_column="0" .../>  <!-- izquierda -->
        <Button android:id="@+id/btnAlfabetico" android:layout_column="2" .../>  <!-- derecha -->
    </TableRow>
</TableLayout>
```

3.6 Ordenar los planetas

Los datos de datos.txt están copiados en ListadoPlanetas.java como tres arrays. Primero se juntan en objetos:

```
ArrayList<ModeloPlaneta> listaPlanetas = new ArrayList<ModeloPlaneta>();
for (int i = 0; i < planetas.length; i++)
    listaPlanetas.add(new ModeloPlaneta(planetas[i], distancias[i], diametros[i]));

if (criterio.equals(CRITERIO_DISTANCIA))
    Collections.sort(listaPlanetas, new Comparator<ModeloPlaneta>() {
        public int compare(ModeloPlaneta p1, ModeloPlaneta p2) {
            return Long.compare(p1.getDistancia(), p2.getDistancia());
        }
    });
// tamaño: Double.compare(p1.getDiametro(), p2.getDiametro())
// alfabético: p1.getNombre().compareToIgnoreCase(p2.getNombre())

if (!ascendente) Collections.reverse(listaPlanetas);
```

Por qué una clase modelo: si ordenas solo el array de nombres, las distancias y los diámetros se quedan en su sitio y los datos se descuadran. Con objetos, al ordenar se mueven los tres datos juntos.

Por qué Long.compare y Double.compare: las distancias son long y pasan del límite de int. Restarlas y convertir a int (como con los puntos del Six Nations) desbordaría y ordenaría mal.

Después se convierte cada planeta en el texto de su fila ("Mercurio: 57909175 km", "Mercurio: 4879.0 km" o solo "Mercurio") y se usa un **ArrayAdapter<String>**. Los diámetros son double, por eso salen con ".0", igual que en las capturas del enunciado.

3.7 Menú y SharedPreferences

```
// onCreate (versión con SP)
preferencias = getSharedPreferences("preferencias_planetas", Context.MODE_PRIVATE);
ascendente = preferencias.getBoolean("ascendente", true);   // true si es la primera vez

// onOptionsItemSelected
if (item.getItemId() == R.id.itemAscendente)  ascendente = true;
if (item.getItemId() == R.id.itemDescendente) ascendente = false;
item.setChecked(true);
SharedPreferences.Editor editor = preferencias.edit();
editor.putBoolean("ascendente", ascendente);
editor.apply();
```

En **onCreateOptionsMenu** se marca la opción activa con **menu.findItem(...).setChecked(true)**, así al abrir el menú se ve cuál está elegida.

Versión sin SP: la variable ascendente solo vive en memoria; al cerrar la app vuelve a ascendente.

3.8 Antes de entregar

- Las imágenes son provisionales: sustitúyelas por las del profesor con los mismos nombres (planetas_fondo.jpg y planetas_splashscreen.jpg en res/drawable).
- Build, Clean Project antes de entregar.

Parte 4. Chuleta rápida: qué usar para cada cosa

Quiero...	Uso	Proyecto de ejemplo
Responder a un botón	setOnClickListener (lambda o clase anónima) o android:onClick	CALCULADORAV10_local
Varios botones, un método	implements View.OnClickListener + v.getId()	CalculadoraProgramatica3
Opciones compatibles	CheckBox + OnCheckedChangeListener	HotelComidas
Opción única entre varias	RadioButton dentro de RadioGroup	Hamburgueseria
Encender/apagar algo	Switch o ToggleButton	Toggle_SwitchButton
Elegir de una lista desplegable	Spinner + ArrayAdapter + onItemSelected	07SpinnerPaises
Lista de textos	ListView + ArrayAdapter	ListViewDeStrings
Lista con filas complejas	Modelo + adaptador propio (getView) + item.xml	ListViewAdaptadorPersonalizado
Refrescar una lista tras cambiar datos	adaptador.notifyDataSetChanged()	Six Nations
Ordenar una lista	Collections.sort + Comparator; Collections.reverse para invertir	Six Nations, Planetas
Abrir otra pantalla	Intent explícito + startActivity	12IntentExplicita
Pasar datos a otra pantalla	putExtra / getIntent().getExtras()	12IntentExplicita
Abrir una web	Intent implícito ACTION_VIEW + Uri.parse	07SpinnerPaises
Cerrar la app	finishAffinity()	PanelInicioSesion
Cerrar solo esta pantalla	finish()	DetallePersona, Splash
Pedir una hora	TimePickerDialog + Calendar	9TimePickerDialog
Formulario en un diálogo	AlertDialog.Builder + setView(layout inflado)	Six Nations
Confirmar una acción	AlertDialog con setPositiveButton / setNegativeButton	Six Nations
Menú de la barra superior	res/menu + onCreateOptionsMenu + onOptionsItemSelected	MenuAplicacion
Menú al mantener pulsado	registerForContextMenu + onCreateContextMenu + onContextItemSelected	Six Nations
Guardar datos al cerrar la app	SharedPreferences: edit(), putX(), apply() / getX(clave, defecto)	Six Nations, Planetas
Pantalla de bienvenida	Actividad LAUNCHER + Thread con sleep + Intent + finish()	Planetas
Formulario en filas/columnas	TableLayout + TableRow	TableLayoutFormulario
Botón flotando sobre una lista	FrameLayout + layout_gravity="bottom|end"	Six Nations
Varios idiomas	values-en, values-fr-rFR... + @string/	11Internacionalizacion
Bloquear la orientación	setRequestedOrientation(...)	OrientacionLayoutBloqueo
Validar un formulario	trim().isEmpty() + Toast; try/catch NumberFormatException	TableLayoutFormulario

Quiero...	Uso	Proyecto de ejemplo
Imagen de un país/opción	Array de R.drawable paralelo al de nombres + setImageResource	07SpinnerPaises
Icono del propio Android	@android:drawable/nombre (ej. ic_input_add)	Six Nations

Errores típicos a evitar

- Crear una Activity y no declararla en el **AndroidManifest.xml**: la app se cierra al abrirla.
- Usar un tema **NoActionBar** y esperar ver el menú de la barra.
- Llamar a findViewById antes de setContentView: devuelve null.
- En un adaptador o diálogo, usar el findViewById de la Activity en lugar del de la vista inflada.
- Cambiar la lista de datos y olvidar **notifyDataSetChanged()**: la pantalla no se actualiza.
- Olvidar **apply()** o **commit()** en SharedPreferences: no se guarda nada.
- Guardar IDs de R.drawable en SharedPreferences: guarda la posición en el array.
- Hacer sleep en el hilo principal: congela la app. Para esperar, usa un Thread.
- Entregar sin **Clean Project**: el proyecto pesa mucho más por las carpetas build.